Ahorro de Tokens — Repos y Estrategias (Seawolf Studio)

- INFORME DE TEMA: Repos y Estrategias para AHORRAR TOKENS
  - 0. RESUMEN EJECUTIVO — 4 frentes de ahorro
  - 1. ⚡ AHORRO INMEDIATO (cero instalación) — LO PRIMERO
  - 2. 🧩 REPOS VERIFICADOS (bajo tus estándares)
  - 3. LAS 6 ESTRATEGIAS (por mecanismo)
  - 4. RECOMENDACIÓN — PLAN DE ADOPCIÓN
  - 5. HONESTIDAD BRUTAL
  - 6. FUENTES

INFORME DE TEMA: Repos y Estrategias para AHORRAR TOKENS

Fecha: 2026-09-30 · **Clasificación:** Uso Interno — Seawolf Studio **Elaborado por:** Bellion (Gran Comandante) · **Encargado por:** Monarca (Key) **Objetivo:** Reducir el consumo de tokens por cualquier vía. Verificado contra GitHub API (licencia/estrellas/fechas) el 2026-09-30. **Estándares del Monarca aplicados:** MIT (innegociable) · ≥1000★ · deployable como servicio · antigüedad ≥6 meses. > **Nota honesta:** en esta categoría casi nada es "sitio web" — son **librerías**, **pasarelas (gateways)** o **proxies**. Marco cada uno con su tipo real.

---

0. RESUMEN EJECUTIVO — 4 frentes de ahorro

Frente	Ahorro	Esfuerzo
1. Lo que YA tienes instalado (skills Caveman + caché de OpenRouter)	Alto, inmediato	⚡ Cero
2. Compresión de prompts (LLMLingua)	2×–20× en el *input*	Bajo
3. Caché semántica (GPTCache / gateway)	Reutiliza respuestas → ~90% en repetidos	Bajo
4. Memoria + enrutado + RAG (mem0, semantic-router, RAG)	Reducción estructural de contexto	Medio

---

1. ⚡ AHORRO INMEDIATO (cero instalación) — LO PRIMERO

1.1 Ya tienes las skills **CAVEMAN** instaladas (¡y no las usas!)

Tu Hermes trae una familia de skills de compresión de tokens **sin coste**: | Skill | Qué hace | |———-|————-| | `caveman` | Modo de comunicación ultra-comprimido: corta el **output** a lo esencial | | `caveman-compress` | Comprime archivos de memoria (CLAUDE.md, todo lists) | | `caveman-stats` | **Muestra tu uso REAL de tokens y el ahorro estimado** | | **caveman-commit / caveman-review / caveman-help / cavecrew** | Variantes para commits, review y ayuda |

👉 **Acción inmediata:** activar `caveman` en sesiones largas y correr `caveman-stats` para medir.

1.2 Caché de prompt del proveedor (OpenRouter)

OpenRouter soporta **prompt caching** (reutiliza el prefijo del prompt: system + contexto). El proveedor reporta hasta **~90% de descuento** en tokens de entrada repetidos y **~80% menos latencia**. Como ya pagas por uso en OpenRouter, **verificar que la caché esté activa** es el ahorro #1.

1.3 Compresión de contexto de Hermes

Hermes ya comprime el contexto automáticamente en sesiones largas. **Protocolo del Monarca ya definido:** cerrar sesión → diario → nueva sesión. Aplicarlo religiosamente evita arrastrar contexto muerto.

---

2. 🧩 REPOS VERIFICADOS (bajo tus estándares)

Repo	★	Lic.	Año	Tipo	Ahorro	¿Cumple?
microsoft/LLMLingua	6.755	**MIT**	2023-07	Librería	**Compresión de prompt 2×–20×** (quita tokens de bajo valor)	✅ (es librería, no web)
zilliztech/GPTCache	8.211	**MIT**	2023-03	Librería/proxy	**Caché semántica** — responde sin llamar al modelo	✅ (librería)
stanfordnlp/dspy	38.638	**MIT**	2023-01	Librería	Optimiza prompts (menos tokens, mejores respuestas)	✅ (librería)
aurelio-labs/semantic-router	3.947	**MIT**	2023-10	Librería	Enruta a **modelo barato/local** según la consulta	✅
Portkey-AI/gateway	13.189	**MIT**	2023-08	**Gateway (deployable)**	Caché + enrutado + guardrails en un proxy	✅ **deployable**
openai/tiktoken	19.457	**MIT**	2022-12	Librería	**Medir** tokens (base para optimizar)	✅
microsoft/PromptWizard	4.018	**MIT**	2024-05	Librería	Optimización de prompts agent-driven	✅
lemony-ai/cascadeflow	3.964	MIT	2025-10	Runtime	**Cascada**: empieza barato, escala solo si hace falta	✅
langchain / llama_index / langgraph	147k / 52k / 43k	MIT	2022-23	Frameworks	**RAG**: recuperar solo lo relevante en vez de todo el contexto	✅
~~BerriAI/litellm~~	61.259	**NO MIT puro** (open-core: dir `enterprise/`)	2023	Gateway	Caché + enrutado	❌ **no MIT puro**
~~n8n~~	—	fair-code	—	—	—	❌

Patrón clave: de tu lista de estándares, **la mayoría son librerías**, no webs. Los que sí son **deployables como servicio** son los **gateways**: **Portkey** (MIT puro) y el descartado litellm (no MIT).

---

3. LAS 6 ESTRATEGIAS (por mecanismo)

#	Estrategia	Herramienta (MIT)	Ahorro
1	**Compresión de prompt** (borrar tokens de bajo valor)	**LLMLingua**	2×–20× input
2	**Caché semántica** (respuestas a preguntas similares)	**GPTCache**	~90% en repetidos
3	**Caché de proveedor** (KV cache del prefijo)	**OpenRouter** (ya activo)	~90% input repetido
4	**Optimización de prompt** (menos palabras, misma tarea)	**DSPy / PromptWizard**	20–60%
5	**Enrutado inteligente** (barato→caro bajo demanda)	**semantic-router / cascadeflow**	50–80% coste
6	**Memoria + RAG** (no meter todo el historial)	**mem0** + RAG	~90% contexto

---

4. RECOMENDACIÓN — PLAN DE ADOPCIÓN

Semana 1 (coste cero): 1. Activar **skill** `caveman` en sesiones largas + correr `caveman-stats` para ver el gasto real. 2. **Verificar la caché de prompt en OpenRouter** (el ahorro más grande y ya disponible). 3. Aplicar el protocolo de compactación de sesión ya definido (Diario → nueva sesión).

Semana 2 (instalación ligera): 4. Montar **Portkey Gateway** (MIT, deployable) delante de OpenRouter → caché semántica + enrutado en un proxy. 5. Añadir **LLMLingua** para comprimir prompts largos (RAG, historial).

Semana 3 (estructural): 6. Integrar **mem0** (corta ~90% del contexto arrastrado) + RAG (recuperar solo lo relevante).

---

5. HONESTIDAD BRUTAL

1. **La mayoría de "repos para ahorrar tokens" NO son webs** — son librerías. Tus estándares (web-deployable) solo los cumplen los **gateways**. Lo digo por transparencia, no para inflar la lista.
2. **La compresión de prompt puede degradar respuestas** si borra información relevante (LMLingua pierde ~1,5 pp en benchmarks). Usar con ratio moderado en tareas críticas.
3. **El ahorro #1 es gratis y ya lo tienes:** skill `caveman` + caché de OpenRouter + protocolo de sesión. Empieza por ahí.
4. **litellm NO es MIT puro** (open-core). Para el criterio innegociable, usar **Portkey Gateway (MIT)**.

---

6. FUENTES

- GitHub Search/Repos API (licencia/estrellas/fechas) — 2026-09-30.
- PointFive “Prompt Compression Tools 2026”; llmlingua.com; premai.io (cost optimization); thelec.net (caching vs compression).
- Skills locales de Hermes: familia `caveman`.

Fin del informe de tema. Listo para auditoría de Beru y archivo en `diario-de-sombras/`.